



Fungsi dan Modularitas

Parameter • Return Value • Scope Variabel

Algoritma Pemrograman • IF25-11001 • 2 SKS Teori

Institut Teknologi Sumatera — Teknik Informatika

Review Pertemuan 12

Sorting — dan transisi ke modularitas

Bubble Sort dan Selection Sort = $O(n^2)$

Dua algoritma sorting dasar. Bubble: swap adj, stabil. Selection: cari min, sedikit swap.



PBL#2 Data Analyzer — SELESAI!

Statistik + Searching + Sorting dalam satu program. Tapi kodenya sudah mulai PANJANG...

Masalah: kode panjang = sulit dikelola!

Hari ini: FUNGSI — pecah program besar menjadi modul-modul kecil yang reusable!

Capaian Pembelajaran Pertemuan 13

Sub-CPMK 13.1

C2 – Menjelaskan

Menjelaskan konsep modularitas, fungsi, parameter, dan return value

CPMK0609

Sub-CPMK 13.2

C3 – Menerapkan

Merancang pseudocode fungsi dengan parameter input dan return value

CPMK0609

Sub-CPMK 13.3

C3 – Menerapkan

Melakukan trace pemanggilan fungsi dan menganalisis scope variabel

CPMK0609

Outline Pertemuan Hari Ini

	00-10	Review dan Hook	Recap sorting + kenapa perlu fungsi
	10-25	Konsep Fungsi	Modularitas, DRY, anatomi fungsi
	25-40	Parameter dan Return	Input, output, return value
	40-55	Scope Variabel	Lokal vs global, pass by val/ref
	55-70	Workshop Trace	Trace pemanggilan fungsi di kertas
	70-80	Algorithm Detective	Bug: scope error dan missing return
	80-90	PBL#3 Kick-off	Student Record System — modular!
	90-100	Exit Ticket dan Tugas	Assessment + homework



Pertanyaan Pemantik

*"Program Data Analyzer sudah 200 baris.
Bagaimana jika nanti 2000 baris? 20.000 baris?"*

*"Kode menghitung rata-rata ditulis 5 kali di tempat berbeda.
Bagaimana jika ada bug — harus fix di 5 tempat?"*

*"Bagaimana tim 5 orang bisa kerja bareng
tanpa saling menimpa kode satu sama lain?"*

Kenapa Perlu Fungsi?

Prinsip DRY: Don't Repeat Yourself!

✗ Tanpa Fungsi (Copy-Paste!)

```
total1 ← 0
UNTUK i ← 0 SAMPAI N1-1
  total1 ← total1 + arr1[i]
rata1 ← total1 / N1

total2 ← 0
UNTUK i ← 0 SAMPAI N2-1
  total2 ← total2 + arr2[i]
rata2 ← total2 / N2

total3 ← 0
... (kode SAMA diulang lagi!)
```

✓ Dengan Fungsi (Tulis Sekali!)

```
FUNGSI hitungRata(arr, N) : real
  total ← 0
  UNTUK i ← 0 SAMPAI N-1
    total ← total + arr[i]
  RETURN total / N
AKHIR FUNGSI

rata1 ← hitungRata(arr1, N1)
rata2 ← hitungRata(arr2, N2)
rata3 ← hitungRata(arr3, N3)
```

4 Keuntungan Fungsi

Reusable

Tulis sekali, panggil berkali-kali

Readable

Kode jadi pendek dan mudah dibaca

Debuggable

Bug? Fix di 1 tempat, langsung fix semua

Teamwork

Bagi tugas: tiap orang bikin fungsi sendiri

Anatomi Fungsi dalam Pseudocode

Setiap fungsi punya: nama, parameter, tipe return, body

```
FUNGSI hitungMaks(arr, N) : integer
maks ← arr[0]
UNTUK i ← 1 SAMPAI N-1
    JIKA arr[i] > maks MAKA
        maks ← arr[i]
    AKHIR JIKA
AKHIR UNTUK
RETURN maks
AKHIR FUNGSI

hasil ← hitungMaks(data, 5)
OUTPUT hasil
```

Komponen Fungsi

- | | |
|-----------------------|--|
| 1. Nama | Kata kerja deskriptif:
hitungMaks, cariMin, sortArray |
| 2. Parameter | Data INPUT yang diterima.
Bisa 0, 1, atau banyak. |
| 3. Tipe Return | Tipe data KELUARAN.
Void = tidak mengembalikan. |
| 4. Body | Logika utama fungsi.
Hitung, proses, olah data. |
| 5. RETURN | Kembalikan hasil ke pemanggil.
Maksimal 1 return value. |

Fungsi vs Prosedur

Fungsi: mengembalikan nilai (RETURN). Contoh: hitungMaks(arr, N) : integer → return 17

Prosedur: TIDAK mengembalikan nilai (void). Contoh: cetakArray(arr, N) → hanya cetak, no return

Parameter dan Return Value

Input → Proses → Output

Fungsi dengan Return

```
FUNGSI luasSegitiga(alas, tinggi) : real
  RETURN (alas * tinggi) / 2
AKHIR FUNGSI

hasil ← luasSegitiga(10, 5)
OUTPUT hasil
```

Parameter: alas, tinggi
Return: real (bilangan pecahan)
Pemanggilan: simpan di variabel

Fungsi Boolean

```
FUNGSI adalahGenap(n) : boolean
  RETURN (n MOD 2 == 0)
AKHIR FUNGSI

JIKA adalahGenap(7) MAKA
  OUTPUT "Genap"
SELAINNYA
  OUTPUT "Ganjil"
```

Parameter: n (integer)
Return: boolean (true/false)
Bisa langsung di JIKA

Prosedur (Void)

```
PROSEDUR cetakArray(arr, N)
  UNTUK i ← 0 SAMPAI N-1
    OUTPUT arr[i], " "
  AKHIR UNTUK
  OUTPUT newline
AKHIR PROSEDUR

cetakArray(data, 5)
```

Parameter: arr, N
Return: TIDAK ADA (void)
Hanya melakukan aksi (cetak)

Trace: Pemanggilan Fungsi

Ikuti alur eksekusi langkah demi langkah

```
FUNGSI kuadrat(x) : integer
  RETURN x * x
AKHIR FUNGSI

FUNGSI jumlahKuadrat(a, b) : integer
  RETURN kuadrat(a) + kuadrat(b)
AKHIR FUNGSI

hasil ← jumlahKuadrat(3, 4)
OUTPUT hasil
```

Step	Aksi	Return	hasil 1
1	panggil jumlahKuadrat(3, 4)		
2	panggil kuadrat(3)	9	
3	panggil kuadrat(4)	16	
4	return 9 + 16	25	
5	hasil ← 25		25
6	OUTPUT 25		25

Pola Pemanggilan Fungsi (Call Stack)

1. Program utama memanggil jumlahKuadrat(3, 4)
2. jumlahKuadrat memanggil kuadrat(3) → return 9
3. jumlahKuadrat memanggil kuadrat(4) → return 16
4. jumlahKuadrat menghitung $9 + 16 = 25$, return 25
5. Program utama menerima 25, simpan di hasil
6. Fungsi yang TERAKHIR dipanggil, PERTAMA selesai (LIFO — seperti tumpukan!)

Scope Variabel: Lokal vs Global

Di mana variabel bisa diakses?

```
x ← 10
```

```
FUNGSI tambah(a) : integer
```

```
  x ← a + 5
```

```
  RETURN x
```

```
AKHIR FUNGSI
```

```
hasil ← tambah(3)
```

```
OUTPUT x
```

```
OUTPUT hasil
```

Analisis Scope

x = 10 (baris 1)

Variabel GLOBAL — bisa diakses di mana saja

a (parameter)

Variabel LOKAL di tambah() — hanya ada di dalam fungsi

x di dalam fungsi

Jika lokal: x baru terpisah dari x global.

Jika global: x global berubah!

OUTPUT x

Tergantung: apakah x di fungsi
lokal atau mengubah global?

Aturan Scope

Lokal: variabel di dalam fungsi. Dibuat saat fungsi dipanggil, HILANG saat fungsi selesai. • Global: variabel di luar fungsi. Bisa diakses semua fungsi — tapi HINDARI! Bikin bug susah dilacak. • Best practice: gunakan parameter dan return, HINDARI global!

Pass by Value vs Pass by Reference

Apakah fungsi mengubah variabel ASLI?

Pass by Value (SALINAN)

```
PROSEDUR tambah5(x)
  x ← x + 5
  OUTPUT "di dalam: ", x
AKHIR PROSEDUR

a ← 10
tambah5(a)
OUTPUT "di luar: ", a
```

Output:
di dalam: 15
di luar: 10 ← TIDAK berubah!

x di fungsi = SALINAN dari a.
Mengubah x tidak mengubah a.

Pass by Reference (ASLI)

```
PROSEDUR tambah5(REF x)
  x ← x + 5
  OUTPUT "di dalam: ", x
AKHIR PROSEDUR

a ← 10
tambah5(a)
OUTPUT "di luar: ", a
```

Output:
di dalam: 15
di luar: 15 ← BERUBAH!

REF x = referensi ke a ASLI.
Mengubah x = mengubah a.

Kapan Pakai?

By Value: fungsi TIDAK perlu mengubah data asli (default, lebih aman).

By Reference: fungsi HARUS mengubah data asli (swap, sort array, input dari user). Di C++: gunakan dan (ampersand).

Workshop: Trace Fungsi

Kerjakan di kertas — 10 menit

Soal 1: Trace Fungsi

```
FUNGSI mystery(a, b) : integer
    JIKA a > b MAKA
        RETURN a - b
    SELAINNYA
        RETURN b - a
    AKHIR JIKA
AKHIR FUNGSI

x ← mystery(3, 8)
y ← mystery(10, 4)
z ← mystery(x, y)
OUTPUT x, y, z
```

Pertanyaan:

- a) Apa yang dilakukan fungsi mystery?
(Jelaskan dengan 1 kalimat)
- b) Trace: berapa nilai x, y, z?
- c) Apa output akhirnya?

Soal 2: Desain Fungsi

Tulis pseudocode untuk fungsi-fungsi berikut:

- a) hitungFaktorial(n) : integer
Mengembalikan n! (n faktorial)
Contoh: hitungFaktorial(5) = 120
- b) adalahPrima(n) : boolean
Mengembalikan true jika n prima
Contoh: adalahPrima(7) = true
- c) hitungVokal(str) : integer
Mengembalikan jumlah huruf vokal dalam string str
Contoh: hitungVokal("ITERA") = 3

Jawaban Workshop

Soal 1: mystery = selisih absolut!

Jawaban Soal 1

a) mystery = menghitung SELISIH ABSOLUT
 $|a - b|$ (jarak antara dua bilangan)

b) Trace:

$x = \text{mystery}(3, 8): 3 < 8 \rightarrow \text{return } 8 - 3 = 5$

$y = \text{mystery}(10, 4): 10 > 4 \rightarrow \text{return } 10 - 4 = 6$

$z = \text{mystery}(5, 6): 5 < 6 \rightarrow \text{return } 6 - 5 = 1$

c) OUTPUT: 5, 6, 1

Jawaban Soal 2a: Faktorial

```
FUNGSI hitungFaktorial(n) : integer
    hasil ← 1
    UNTUK i ← 1 SAMPAI n
        hasil ← hasil * i
    AKHIR UNTUK
    RETURN hasil
AKHIR FUNGSI
```

$5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$ ✓

Soal 2b: adalah Prima

```
FUNGSI adalahPrima(n) : boolean
    JIKA n < 2 MAKA RETURN false
    UNTUK i ← 2 SAMPAI n-1
        JIKA n MOD i == 0 MAKA
            RETURN false
    AKHIR UNTUK
    RETURN true
```

Soal 2c: hitungVokal

```
FUNGSI hitungVokal(str) : integer
    count ← 0
    UNTUK i ← 0 SAMPAI LENGTH(str)-1
        JIKA str[i] IN {a,i,u,e,o,A,I,U,E,O}
            count ← count + 1
    AKHIR UNTUK
    RETURN count
```



Algorithm Detective

Temukan bug pada fungsi berikut!

Kasus 1: Missing Return

```
FUNGSI cariMaks(arr, N) : integer
maks ← arr[0]
UNTUK i ← 1 SAMPAI N-1
    JIKA arr[i] > maks MAKA
        maks ← arr[i]
AKHIR FUNGSI
```

Bug: TIDAK ADA RETURN!
Fungsi menghitung maks tapi
tidak mengembalikan hasilnya.

Pemanggil tidak mendapat apa-apa.
hasil ← cariMaks(arr, 5) = ???

Fix: tambahkan RETURN maks
sebelum AKHIR FUNGSI.

Kasus 2: Scope Confusion

```
total ← 100

FUNGSI hitungTotal(arr, N) : integer
    total ← 0
    UNTUK i ← 0 SAMPAI N-1
        total ← total + arr[i]
    RETURN total
AKHIR FUNGSI

OUTPUT total
```

Expected: OUTPUT = hasil hitungTotal
Actual: OUTPUT = 100 (global!)

Bug: hitungTotal dipanggil tapi
hasilnya TIDAK disimpan!
total di fungsi = LOKAL.
OUTPUT total = global (100).

Fix: total ← hitungTotal(arr, N)
OUTPUT total



Aturan Emas Fungsi

Hindari bug yang paling umum

1

Selalu RETURN jika ada tipe return

Fungsi dengan tipe return (integer, real, boolean) HARUS ada RETURN. Tanpa return = bug!

Cek: setiap jalur punya RETURN

2

Simpan hasil pemanggilan!

hasil ← namaFungsi(param). Jangan panggil tanpa simpan jika butuh hasilnya.

x ← fungsi(param)

3

HINDARI variabel global

Global bikin bug susah dilacak. Gunakan parameter dan return. Lokal lebih aman.

Pakai parameter + return

4

Nama fungsi = kata kerja

hitungTotal, cariMaks, adalahPrima. Bukan total, maks, prima. Deskriptif dan jelas.

hitungX, cariX, adalahX

PBL#3: Student Record System — Kick-off!

Deadline P15 — 7% dari nilai akhir

Deskripsi Proyek

Buat program Student Record System yang MODULAR menggunakan fungsi:

- Input data 5 mahasiswa (nama, NIM) menggunakan struct
- Input berulang sebanyak 3 matakuliah (nama, SKS, nilai dalam huruf) dengan struct
- Hitung IPK setiap mahasiswa (fungsi hitungIPK)
- Cari mahasiswa berdasarkan NIM (fungsi cariMahasiswa)
- Urutkan berdasarkan IPK (fungsi sortByIPK)
- Tampilkan laporan (prosedur cetakLaporan)
- Gunakan REKURSI minimal 1 fungsi (P14)

Timeline PBL#3

P13

Desain fungsi modular: hitungIPK, cariMhs, sortByIPK

Milestone 1

P14

Rekursi + Struct: definisi struct Mahasiswa, rekursi

Milestone 2

P15

Review dan integrasi → SUBMIT

Deadline!

Ringkasan Pertemuan 13

Fungsi = Modularitas + DRY

Pecah program besar jadi modul kecil. Tulis sekali, panggil berkali-kali.

Parameter + Return Value

Parameter = input. Return = output. Fungsi void (prosedur) = tanpa return.

Scope: Lokal vs Global

Lokal di dalam fungsi, hilang saat selesai. Hindari global — pakai parameter!



Pass by Value vs Reference

Value = salinan (aman). Reference = asli (mengubah). Default: value.

PBL#3 Student Record System

Kick-off hari ini! Desain fungsi modular. Deadline P15.

Exit Ticket — Pertemuan 13

10 soal • 15 menit • kerjakan di kertas

1

Apa 4 keuntungan menggunakan fungsi? Sebutkan dan jelaskan.

2

Apa perbedaan FUNGSI dan PROSEDUR? Beri contoh masing-masing.

3

Trace: `hitungMaks({3,9,1,7}, 4)`. Return apa?

4

Tulis fungsi `hitungMin(arr, N)` : integer. Pseudocode lengkap!

5

Apa perbedaan pass by value dan pass by reference?

6

$f(x)=x^2$. $g(a,b)=f(a)+f(b)$. Berapa $g(3,5)$?

7

Bug: fungsi return integer tapi tidak ada RETURN. Apa yang terjadi?

8

Apa itu scope variabel? Kenapa hindari global?

9

Tulis fungsi `adalahGenap(n)` : boolean. Pseudocode!

10

Fungsi `swap(REF a, REF b)`. Kenapa harus REF? Apa tanpa REF?

Tugas Mingguan — Fungsi dan Modularitas

Deadline: sebelum Pertemuan 14 • Kerjakan di kertas

Tugas A: Individual — Desain dan Trace Fungsi

Soal 1: Tulis pseudocode fungsi-fungsi berikut:

- (a) `hitungRata(arr, N)` : real — mengembalikan rata-rata array
- (b) `hitungRange(arr, N)` : integer — mengembalikan selisih maks dan min
- (c) `countAboveAvg(arr, N)` : integer — hitung elemen di atas rata-rata
(Hint: panggil `hitungRata` di dalam `countAboveAvg`!)

Soal 2: Diberikan $f(x) = 2x + 1$ dan $g(a, b) = f(a) * f(b)$.

Trace pemanggilan `g(3, 4)`. Tulis call stack!

Soal 3: Refactor program sorting dari P12 menjadi fungsi-fungsi:

`bubbleSort(arr, N)`, `selectionSort(arr, N)`, `cetakArray(arr, N)`.

Tugas B: Kelompok — PBL#3 Milestone 1

Desain pseudocode Student Record System: definisikan fungsi `hitungIPK`, `cariMahasiswa`, `sortByIPK`.

Buat daftar fungsi + parameter + return type. Diskusikan pembagian tugas dalam tim.

Referensi Pertemuan 13

Buku

1. Deitel — C++ How to Program, Bab 6: Functions
2. Gaddis — Starting Out with C++, Bab 6: Functions
3. Cormen (CLRS) — Introduction to Algorithms, Bab 1: Algorithms

Online

1. cppreference.com/functions — C++ Function Reference
2. cs50.harvard.edu — Lecture 1: Functions
3. geeksforgeeks.org/functions-in-cpp — Functions Tutorial



Terima Kasih

IF25-11001 Algoritma Pemrograman

Pertemuan 13: Fungsi dan Modularitas

Program Studi Teknik Informatika
Institut Teknologi Sumatera
2026